FIAP

MBA

Fundamentos de Prompt Engineering, Meta-Prompting e Controle de Contexto

Enterprise Architecture with Generative AI — Aula 01

MBA Executivo em Arquitetura de Soluções — FIAP

Prof. Francisco Carlos Junqueira Junior

FRANCISCO JUNQUEIRA

Gerente de Arquitetura Corporativa

Apaixonado por tecnologia desde que escreveu seu primeiro programa aos 8 anos, Francisco acumula mais de 25 anos de experiência em tecnologia, liderança executiva e transformação digital em setores como saúde, serviços financeiros, indústria farmacêutica e telecomunicações. Atualmente é Gerente de Arquitetura Corporativa e Arquitetura de IA na Orizon.

Engenheiro Eletricista pela **UNICAMP**, com especialização em Microeletrônica pela Berner Fachhochschule (Suíça), MBA pela **FGV** e formação executiva pela **Kellogg School of Management** e **Fundação Dom Cabral**.

Ocupou posições de CIO, Diretor de Tecnologia e Diretor de R&D em grupos multinacionais como Hypercom South America, Prati-Donaduzzi e Nohda Group, liderando equipes multidisciplinares em diferentes países, conduzindo projetos em escala global. Criou a pioneira solução de **Extrato Online de Cartão de Crédito no Brasil** para Itaú-Credicard e contribuiu para a evolução da **API de log** do Java Standard Edition.

Reconhecido com o **Java Duke Award** por sistema de escala massiva aclamado publicamente por James Gosling (criador do Java) na JavaOne, San Francisco 2004.

BLOCO 1 — Fundamentos de Prompt Engineering e Controle de Contexto

Agenda da Aula

FIAP MBA+

Bloco 1 (19:10–21:00)

- Por que Prompt Engineering importa para arquitetos?
- Como LLMs funcionam — mecanismo interno, alucinação e a analogia de Galileu
- 3 problemas adicionais: context rot, overgeneralization, knowledge cutoff
- 5 técnicas de controle de contexto
- Anatomia de um prompt profissional

Bloco 2 (21:15–23:00)

- Meta-Prompting: conceito e fluxo
- 3 exemplos aplicados a arquitetura corporativa
- Antipadrões comuns em prompts
- Exercício guiado: análise comparativa de prompts
- Takeaways e próximos passos

Por que Prompt Engineering Importa para Arquitetos?

FIAP MBA+

O cenário atual

- LLMs estão sendo adotados em escala corporativa: coding assistants, análise de documentação, geração de artefatos
- **O problema:** a maioria dos profissionais interage com LLMs como se fossem buscadores — queries vagas, sem contexto, sem formato
- **O resultado:** respostas genéricas, alucinações não detectadas, decisões baseadas em outputs não confiáveis

O papel do arquiteto

- O arquiteto é o profissional que **define padrões de uso** de IA nas organizações
- Não basta "usar ChatGPT": é preciso **projetar a interação** com a mesma disciplina com que se projeta uma API ou um contrato de integração
- Prompt Engineering é a **interface de comunicação** entre o arquiteto e o LLM

A analogia

> *"Um prompt mal construído para um LLM é como uma user story sem critérios de aceite — vai funcionar, mas ninguém sabe se o resultado é o que se esperava."*

Como LLMs Funcionam: O Mínimo que o Arquiteto Precisa Saber

FIAP MBA+

O que um LLM faz (simplificado)

Recebe uma sequência de tokens (seu prompt + contexto)

Prediz o próximo token com base em probabilidades aprendidas durante o treinamento

Repete até completar a resposta

O mecanismo interno: probabilidade, não verdade

Uma LLM é treinada consumindo uma quantidade colossal de texto produzido pela humanidade: livros, artigos científicos, documentação técnica, fóruns, código-fonte, legislação. Após esse processo, o modelo **conhece tudo** que estava nesses dados. Quando você faz uma pergunta, ele percorre os caminhos mais prováveis dentro dessa base de treinamento para compor a resposta.

A palavra-chave aqui é **provável**, não **correta**.

O modelo não tem um mecanismo para distinguir "*isso é verdade*" de "*isso é o que a maioria dos textos no meu treinamento diria*". Ele responde com o **caminho estatisticamente mais frequente**, que muitas vezes coincide com a resposta correta, mas nem sempre.

> *Quando o modelo não tem dados suficientes para uma resposta confiável, ele não para. Ele continua gerando a continuação mais provável, inventando dados se necessário, porque é exatamente para isso que foi otimizado.*

Por que o modelo "chuta" em vez de dizer "não sei"

Esse comportamento não é um defeito acidental. É consequência direta de como LLMs são treinados:

Como LLMs Funcionam: O Mínimo que o Arquiteto Precisa Saber (cont.)

- **Pré-treinamento:** o modelo aprende a prever o próximo token. Gerar qualquer token é melhor do que gerar nenhum: o sistema nunca é recompensado por "ficar em silêncio"
- **Pós-treinamento (RLHF):** avaliadores humanos classificam respostas. Uma resposta elaborada e confiante quase sempre recebe nota melhor do que "não tenho certeza", mesmo quando a incerteza seria mais honesta
- **Resultado:** o sistema de avaliação **recompensa chutes e penaliza a incerteza honesta**

> *É exatamente como uma prova de múltipla escolha em que deixar a questão em branco vale zero e errar também vale zero, a estratégia ótima é sempre chutar. Na dúvida, a LLM sempre chuta.*

A IA não alucina só porque está "quebrada". Ela alucina porque o sistema inteiro, do treinamento à avaliação, foi projetado para que ela **sempre produza uma resposta**, mesmo quando não deveria.

A analogia de Galileu: quando o "consenso" está errado

Para entender isso de forma visceral, imagine o seguinte experimento mental:

> *Pegamos **todo o conhecimento produzido pela humanidade até o ano de 1630**: todos os livros, textos científicos, tratados religiosos, pensamentos filosóficos, registros acadêmicos, tudo, e treinamos uma LLM exatamente nos mesmos moldes que fazemos hoje: pré-treinamento massivo, pós-treinamento supervisionado, aprendizado por reforço com feedback humano.*

O resultado seria um modelo que funciona **exatamente** como os que usamos hoje: fluente, articulado, confiante, mas com os dados do século XVII.

Problema Fundamental #1: Alucinação

Agora, nesse mesmo período, existe um sujeito chamado **Galileu Galilei** que ficou trancado no seu escritório por anos estudando o sistema solar. Depois de muito estudo, observação e cálculo, ele chega à conclusão de que **a Terra orbita o Sol**, e não o contrário.

Pergunta: se pegarmos essa LLM de 1630 e perguntarmos *"A hipótese de Galileu sobre a Terra não ser o centro do sistema solar está correta?"*, o que ela responde?

Resposta: ela diria, com total confiança e eloquência, que **Galileu está equivocado**. Citaria Aristóteles, Ptolomeu, a física aristotélica, textos teológicos, o consenso acadêmico da época, tudo apontando que o modelo geocêntrico é obviamente correto. Ela provavelmente classificaria Galileu como um pensador excêntrico com ideias perigosas e sem fundamento.

E o modelo não estaria "alucinando" no sentido técnico, ele estaria respondendo com **o caminho mais provável** dentro dos seus dados de treinamento. Acontece **que o consenso inteiro dos dados estava errado**.

Alucinação: o nome técnico para o "chute confiante"

Esse comportamento tem um nome formal: **alucinação**. O LLM gera informações plausíveis porém factualmente incorretas, apresentando-as com a mesma confiança de um fato verificado.

Agora que entendemos o mecanismo, fica claro: alucinação **não é um bug**: é consequência direta do design. O modelo otimiza para fluência e coerência, não para verdade factual. Quando não tem dados suficientes, preenche as lacunas com a continuação mais provável, e apresenta como fato.

Problema Fundamental #1: Alucinação (cont.)

FIAP MBA+

Como isso se manifesta no dia a dia do arquiteto

Cenário	O que o LLM faz
Perguntar sobre uma API específica	Inventa endpoints, parâmetros ou comportamentos que não existem
Solicitar análise de performance	Cita benchmarks inexistentes com números específicos e fontes falsas
Pedir recomendação de biblioteca	Sugere pacotes que não existem ou combina features de pacotes diferentes
Avaliar compliance	Cita regulamentações com artigos e parágrafos inventados

Alucinações são especialmente perigosas quando **parecem precisas** — com números, referências bibliográficas e justificativas elaboradas. **O modelo não avisa que está alucinando**. Apresenta tudo com o mesmo nível de confiança.

O que isso ensina ao arquiteto

Lição	Implicação prática
O LLM reflete o consenso dos dados, não a verdade	Sempre questione se o "consenso" no output é aplicável ao seu contexto
O modelo nunca vai dizer "eu não sei" por conta própria	Você precisa **forçar** a declaração de incerteza via prompt (Explicit Uncertainty)
Respostas confiantes ≠ respostas corretas	A confiança do modelo não é um indicador de acurácia
Dados ausentes são preenchidos, não sinalizados	Se o dado não está no prompt, o modelo inventa — e apresenta como fato

Problema Fundamental #1: Alucinação (cont.)

FIAP MBA+

Implicações arquiteturais

Característica	Implicação para o arquiteto
Predição probabilística	O LLM não "sabe”: ele calcula a continuação mais provável. Respostas plausíveis ≠ respostas corretas
Context window finito	Existe um limite de tokens que o modelo processa por vez (ex: 200K tokens no Claude). Informação fora da janela é ignorada
Sem estado persistente	Cada chamada é independente. O modelo não "lembra" conversas anteriores a menos que sejam incluídas no prompt
Treinamento estático	O conhecimento do modelo tem uma data de corte (knowledge cutoff). Não sabe o que aconteceu depois

Como mitigar (preview — veremos em detalhe nas técnicas a seguir)

- **Grounding:** fornecer dados concretos no prompt reduz o espaço para invenção
- **Explicit Uncertainty:** instruir o modelo a declarar quando não tem certeza transforma chutes em sinalizações
- **Verificação humana:** nunca aceitar outputs críticos sem validação: **o arquiteto decide, o LLM informa**

Problema Fundamental #2: Context Rot

Definição

Context rot (degradação de contexto) ocorre quando a qualidade e relevância do contexto se deterioram à medida que a conversa se estende ou o prompt fica muito longo.

Mecanismo

- LLMs têm um **context window** finito (ex: 200K tokens no Claude Opus, 128K no GPT-4)
- Mesmo dentro da janela, o modelo dá **mais peso** ao início e ao final do contexto ("U-shaped attention")
- Informações no **meio** de contextos muito longos tendem a ser ignoradas ou mal-ponderadas
- Em conversas longas, **instruções iniciais são progressivamente diluídas** por novas mensagens

Sintomas visíveis

- O modelo ”**simplifica**” e “**esquece**” instruções dadas no início da conversa
- Respostas começam a contradizer restrições já definidas
- O formato de saída vai se **degradando** ao longo de múltiplas interações
- O modelo começa a misturar contextos de partes diferentes da conversa

Problema Fundamental #2: Context Rot (cont.)

FIAP MBA+

Impacto para o arquiteto

- **Análises longas e iterativas** (ex: refinar uma proposta arquitetural ao longo de vários turnos) são particularmente vulneráveis
- Pode levar a **inconsistências** dentro do mesmo documento gerado
- Restrições de compliance ou segurança definidas no início podem ser "esquecidas"

Como mitigar

- **Context Bounding:** manter prompts focados e com **escopo bem definido**
- **Sessões curtas e objetivas** em vez de conversas longas e abertas
- **Repetir restrições críticas** em cada interação quando necessário
- **System prompts** para instruções que devem persistir (em APIs)

Problema Fundamental #3: Overgeneralization

Definição

Overgeneralization é quando o LLM produz respostas genéricas e teóricas que ignoram as especificidades do cenário real apresentado.

Por que acontece?

- O modelo foi treinado em **bilhões de textos genéricos**: artigos, documentação, blogs, livros
- A resposta "média" é quase sempre genérica porque é a que se aplica ao maior número de situações no conjunto de treinamento
- **Prompts vagos recebem respostas vagas**: o modelo responde no mesmo nível de especificidade da pergunta

Exemplos

Prompt vago:

```
Qual a melhor arquitetura para um sistema de e-commerce?
```

Resposta overgeneralized (típica):

> *"Microserviços são recomendados para e-commerce por oferecerem escalabilidade, independência de deploy e resiliência. Use API Gateway, Event-Driven Architecture e CQRS para melhor performance..."*

O problema: essa resposta serve para qualquer e-commerce: ignora o tamanho da empresa, time disponível, budget, requisitos regulatórios, stack atual, etc.

Problema Fundamental #3: Overgeneralization (cont.)

FIAP MBA+

Impacto para o arquiteto

- Recomendações genéricas podem ser **tecnicamente corretas mas contextualmente erradas**
- "Microserviços" pode ser a resposta errada para uma startup de 3 pessoas, mas o LLM não sabe disso se você não disser
- Decisões baseadas em overgeneralization => decisões baseadas em "best practices" descontextualizadas

Como mitigar

- **Grounding:** fornecer dados concretos e específicos do cenário
- **Context Bounding:** delimitar exatamente o escopo da análise
- **Restrições explícitas:** instruir o modelo a não generalizar

Quantas vezes vocês já viram alguém adotar microserviços porque leu que é 'best practice', sem considerar o contexto? O LLM faz exatamente a mesma coisa, a menos que você force o contrário.

Problema Fundamental #4: Knowledge Cutoff

Definição

Knowledge cutoff é a data limite do conjunto de treinamento do modelo. O LLM não possui conhecimento de eventos, tecnologias ou mudanças ocorridas após essa data.

Exemplos de impacto

Área	Risco
Frameworks e bibliotecas	Recomenda versões desatualizadas ou APIs depreciadas
Cloud services	Não conhece novos serviços ou mudanças de pricing
Regulamentações	Pode não refletir mudanças recentes em LGPD, GDPR, etc.
Vulnerabilidades	Não conhece CVEs descobertos após o cutoff
Padrões arquiteturais	Pode não refletir evolução recente em práticas (ex: novos padrões de MCP, agentic workflows)

Cutoffs dos principais modelos (referência)

```
Claude (Opus/Sonnet)          Início de 2025
GPT-4o                        Outubro 2023
Gemini 1.5                    Meados de 2024
```

Nota: estes valores mudam com novas versões. Sempre verificar documentação oficial.

Problema Fundamental #4: Knowledge Cutoff (cont.)

FIAP MBA+

Como mitigar

- **Grounding com dados atuais:** fornecer documentação atualizada no prompt
- **RAG (Retrieval Augmented Generation):** tema da Aula 2: conectar o LLM a bases de conhecimento atualizadas
- **Declaração explícita:** informar ao modelo qual é o estado atual relevante
- **Explicit Uncertainty:** instruir o modelo a sinalizar quando a resposta pode estar desatualizada

Esses 4 problemas: alucinação, context rot, overgeneralization e knowledge cutoff, não são bugs. São características fundamentais de como LLMs funcionam. A boa notícia é que temos técnicas específicas para mitigar cada um deles

Quadro Resumo: Problemas Fundamentais

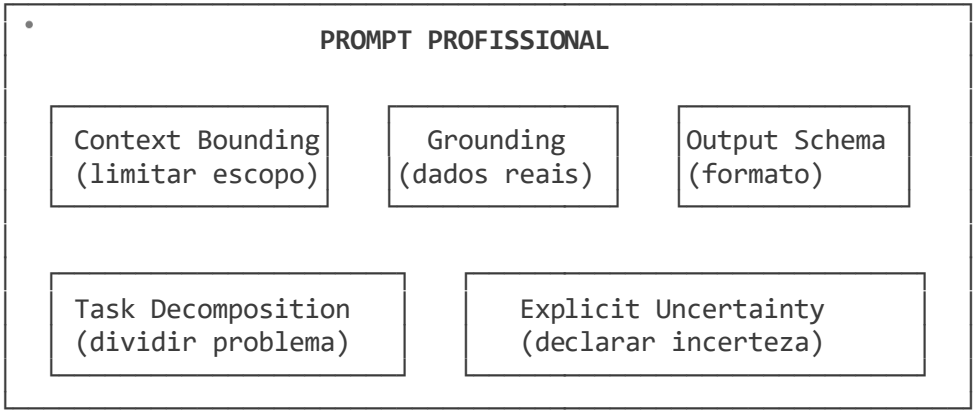
Problema	O que é	Risco para o arquiteto	Mitigação primária
Alucinação	Gera informações falsas com confiança	Decisões baseadas em dados inventados	Grounding + Explicit Uncertainty
Context Rot	Contexto se degrada em conversas longas	Instruções e restrições são ignoradas	Context Bounding + sessões focadas
Overgeneralization	Respostas genéricas, sem considerar cenário	Recomendações descontextualizadas	Grounding + restrições explícitas
Knowledge Cutoff	Não conhece dados após data de treinamento	Versões, APIs e regulamentações desatualizadas	RAG + dados atuais no prompt

Mensagem-chave

*Esses problemas não são resolvidos com "modelos melhores": são mitigados com **prompts melhores**. É por isso que Prompt Engineering é uma competência profissional, não um truque.*

Técnicas de Controle de Contexto: Visão Geral

As 5 técnicas fundamentais



Cada técnica endereça um ou mais dos problemas fundamentais. O poder está na **combinação**: prompts profissionais usam todas simultaneamente.

Técnica 1: Context Bounding

O que é

Delimitar explicitamente o escopo da informação e da análise que o LLM deve considerar.

Por que importa

- Sem bounding, o modelo considera **todo o espaço de possibilidades** e dá respostas genéricas
- Context Bounding **força o modelo a se restringir** ao cenário específico
- Mitiga **overgeneralization** e **context rot**

Como aplicar

```
Escopo temporal         "Considere apenas os próximos 12 meses"
Escopo tecnológico      "Analise apenas no contexto de Azure PaaS"
Escopo organizacional   "Para uma equipe de 5 desenvolvedores, sem SREs dedicados"
Restrição negativa      "Não considere soluções que exijam Kubernetes"
Foco específico         "Analise apenas o aspecto de latência, ignore custos neste momento"
```

Técnica 1: Context Bounding (cont.)

FIAP MBA+

Padrão de uso no prompt

```
[CONTEXTO]: Analise APENAS o cenário descrito abaixo.
Não generalize para outros contextos ou faça suposições
além dos dados fornecidos.

- Escopo: migração lift-and-shift para Azure
- Horizonte: próximos 12 meses
- Restrição: budget máximo de R$ 500k/ano
```

Antipattern

❌ "Me ajude a pensar sobre migração para cloud"
(sem escopo → resposta genérica sobre cloud migration em geral)

✅ "Avalie a viabilidade de migração lift-and-shift para Azure, considerando uma fintech com 40 VMs on-premises, budget de R$ 500k/ano, nos próximos 12 meses"

É a técnica mais importante para arquitetos. Quando vocês fazem uma RFP, vocês definem escopo ou pedem 'uma solução genérica'? O mesmo princípio vale para prompts.

Técnica 2: Grounding

O que é

Fornecer dados concretos e factuais no próprio prompt, em vez de depender do conhecimento interno do modelo.

Por que importa

- O LLM não conhece **seus** dados, **sua** infraestrutura, **seus** requisitos
- Sem grounding, o modelo **inventa** dados plausíveis → alucinação
- Com grounding, o modelo **raciocina sobre** os dados fornecidos → análise útil

O que incluir como grounding

```
Infraestrutura        "40 VMs, Oracle DB 12c, datacenter São Paulo"
Métricas              "SLA 99,9%, latência p99 < 200ms, 500 req/s no pico"
Restrições            "Budget R$ 500k/ano, equipe de 3 DevOps, sem expertise em K8s"
Requisitos regulatór  "Dados regulados por LGPD, armazenamento obrigatório em território
Contexto de negócio   "Fintech, série B, plano de crescimento 3x em 2 anos"
Stack atual           ".NET 6, SQL Server 2019, Azure DevOps, deploys quinzenais"
```

Técnica 2: Grounding (cont.)

Padrão de uso no prompt

```
[CONTEXTO]:
- Empresa: fintech com 200 funcionários, dados regulados por LGPD
- Infraestrutura atual: datacenter on-premises, 40 VMs, Oracle DB 12c
- Workloads críticos: sistema de pagamentos (SLA 99,9%), analytics batch noturno
- Restrições: budget máx R$ 500k/ano em cloud, equipe de 3 DevOps
```

Regra de ouro

*"Se um dado é relevante para a decisão, ele deve estar no prompt. Se não está no prompt, **o modelo vai inventar ou ignorar**."*

Reflexão

Vocês dariam um briefing vago a uma McKinsey e esperariam uma recomendação precisa? O LLM é o mesmo: a qualidade do output é proporcional à qualidade do input (SISO).

Técnica 3: Output Schema

O que é

Definir explicitamente o formato, estrutura e campos que a resposta deve ter.

Por que importa

- Sem schema, o modelo escolhe um formato arbitrário — difícil de comparar, validar ou integrar
- Com schema, a resposta é **padronizada, parseable e auditável**
- Essencial para **automação**: quando o output do LLM alimenta outro sistema

Formatos comuns

JSON	Quando o output será consumido programaticamente ou integrado a pipelines
Tabela Markdown	Para comparações, matrizes de decisão, análises side-by-side
ADR (Architecture Decision Record)	Para documentar decisões arquiteturais
Lista estruturada	Para recomendações priorizadas, checklists, risk assessments
YAML	Para configurações, definições de infraestrutura

Técnica 3: Output Schema (cont.)

Exemplo: Output Schema em JSON

```
[FORMATO DE SAÍDA — JSON]:
{
  "viabilidade": "alta|média|baixa",
  "custo_estimado_anual": "R$ X",
  "riscos_principais": [
    {
      "risco": "descrição",
      "probabilidade": "alta|média|baixa",
      "impacto": "1-5",
      "mitigacao": "ação proposta"
    }
  ],
  ...
```

Exemplo: Output Schema em ADR

```
[FORMATO — ADR]:
Status: [Proposto|Aceito|Depreciado]
Contexto: [2-3 linhas descrevendo o problema de decisão]
Opções consideradas:
  1. Opção A — prós e contras para ESTE cenário
  2. Opção B — prós e contras para ESTE cenário
Decisão: [opção escolhida e justificativa em 3-5 linhas]
Consequências: [positivas e negativas esperadas]
```

Output Schema não é só "formatação bonita": é controle de qualidade.

Quando o modelo precisa preencher campos específicos, ele é forçado a abordar cada ponto.

Se você pede "analise riscos", pode receber um parágrafo vago. Se pede um JSON com campos de probabilidade, impacto e mitigação para cada risco, o modelo é obrigado a ser específico.

Técnica 4: Task Decomposition

O que é

Dividir um problema complexo em subtarefas menores e sequenciadas, instruindo o modelo a resolver cada parte antes de sintetizar.

Por que importa

- LLMs têm dificuldade com **raciocínio multi-step complexo** em uma única passada
- Problemas grandes → respostas superficiais ou que pulam etapas
- Decomposição força o modelo a **abordar cada aspecto** com a profundidade necessária

Padrões de decomposição

Sequencial (chain):

```
[TAREFA]:
1. Primeiro, liste os componentes do sistema afetados pela mudança
2. Para cada componente, identifique o risco de impacto (alto/médio/baixo)
3. Para os riscos altos, proponha uma estratégia de mitigação
4. Ao final, consolide em uma recomendação de go/no-go
```

Paralela (por dimensão):

```
[TAREFA]: Avalie esta proposta em 4 dimensões independentes:
- Custo: TCO projetado em 3 anos
- Performance: impacto em latência e throughput
- Segurança: novos vetores de ataque introduzidos
- Operação: complexidade operacional para o time atual
```

Técnica 4: Task Decomposition (cont.)

FIAP MBA+

Hierárquica (zoom in):

```
[TAREFA]:
- Visão macro: resuma em 3 linhas a viabilidade geral
- Análise por camada: detalhe riscos para frontend, backend, dados e infra
- Deep-dive: para a camada de maior risco, proponha arquitetura alternativa
```

Analogia para arquitetos

> *Task Decomposition para prompts é como C4 Model para arquitetura: Context → Container → Component → Code. Você não tenta resolver tudo em um único diagrama.*

Técnica 5: Explicit Uncertainty

O que é

Instruir o modelo a declarar explicitamente quando não tem certeza ou quando faltam dados para uma análise confiável.

Por que importa

- LLMs **nunca dizem espontaneamente** "não sei": sempre geram uma resposta
- Sem essa instrução, **incerteza é mascarada como confiança**
- Com essa instrução, o arquiteto sabe **onde precisa validar** e **onde pode confiar**

Como aplicar

No prompt:

```
[RESTRIÇÃO]: Se faltar informação para uma análise confiável,
liste em "incertezas" em vez de assumir dados não fornecidos.
Indique claramente quando uma recomendação é baseada em premissas
que precisam ser validadas.
```

No output schema:

```
{
  "analise": "...",
  "confianca": "alta|média|baixa",
  "incertezas": ["lista de pontos onde faltam dados"],
  "premissas_assumidas": ["lista de suposições feitas"]
}
```

Técnica 5: Explicit Uncertainty (cont.)

FIAP MBA+

Variações úteis

Instrução	Efeito
"Se não tiver dados suficientes, diga 'DADOS INSUFICIENTES'"	Resposta binária: sabe ou não sabe
"Classifique sua confiança em cada recomendação como alta/média/baixa"	Graduação de confiança
"Separe fatos verificáveis de inferências suas"	Transparência na cadeia de raciocínio
"Liste premissas assumidas que precisam ser validadas pelo time"	Acionável: gera follow-up concreto

O resultado

Quando o modelo diz "não sei" ou "preciso de mais dados", isso é **mais valioso** do que uma resposta confiante mas errada. Uma incerteza explícita é um **sinal de qualidade**, não de falha.

Essa técnica muda radicalmente a confiabilidade do output

Quadro Integrado: Técnicas × Problemas

FIAP MBA+

Matriz de cobertura

Técnica	Alucinação	Context Rot	Overgeneralization	Knowledge Cutoff
Context Bounding	◐	●	●	◐
Grounding	●	○	●	●
Output Schema	◐	◐	◐	○
Task Decomposition	◐	●	◐	○
Explicit Uncertainty	●	○	○	●

● = mitigação forte | ◐ = mitigação parcial | ○ = sem efeito direto

Mensagem-chave

- Nenhuma técnica sozinha resolve todos os problemas
- **Prompts profissionais combinam todas as 5 técnicas**
- A qualidade do prompt é proporcional à disciplina na aplicação dessas técnicas

Anatomia de um Prompt Profissional

Estrutura recomendada

```
[PAPEL]:        ← Define quem o modelo deve "ser"
   Persona com expertise específica e restrições de comportamento

[CONTEXTO]:     ← Grounding + Context Bounding
   Dados concretos do cenário
   Escopo delimitado
   Restrições explícitas

[TAREFA]:       ← O que deve ser feito (com Task Decomposition)
   Subtarefas sequenciadas ou dimensões de análise

[FORMATO]:      ← Output Schema
   Estrutura exata da resposta esperada
   Campos obrigatórios

[RESTRIÇÃO]:    ← Explicit Uncertainty + Context Bounding
   O que o modelo NÃO deve fazer
   Quando declarar incerteza
   Limites de escopo
```

Checklist rápido

Antes de enviar um prompt profissional, verifique:

- [ ] O papel está definido com expertise específica?
- [ ] O contexto tem dados concretos (não genéricos)?
- [ ] O escopo está delimitado (o que está IN e OUT)?
- [ ] A tarefa está decomposta em passos ou dimensões?
- [ ] O formato de saída está especificado?
- [ ] Há instrução para declarar incertezas?

Recapitulação visual

```
COMO LLMs FUNCIONAM          PROBLEMAS                  TÉCNICAS

┌──────────────────┐    ┌──────────────────────┐    ┌──────────────────────┐
│ Predição         │    │ * Alucinação         │    │ * Context Bounding   │
│ probabilística,  │──>>│ * Context Rot        │──>>│ * Grounding          │
│ não verdade      │    │ * Overgeneralization │    │ * Output Schema      │
│ (Galileu, 1630)  │    │ * Knowledge Cutoff   │    │ * Task Decomposition │
└──────────────────┘    └──────────────────────┘    │ * Explicit Uncertainty │
                                                    └──────────────────────┘
```

BLOCO 2 — Meta-Prompting

Recapitulação Rápida e Transição

FIAP MBA+

O que vimos no Bloco 1

4 problemas fundamentais que todo arquiteto enfrenta ao usar LLMs

5 técnicas de controle de contexto para mitigá-los

Anatomia de um prompt profissional template estruturado

O que veremos agora

- **Meta-Prompting:** como automatizar a criação de prompts estruturados
- **3 exemplos completos** aplicados a cenários de arquitetura corporativa
- **Antipadrões** comuns (o que NÃO fazer)

Meta-Prompting: Conceito

Definição

Meta-Prompting é a técnica de usar um LLM para gerar prompts estruturados e otimizados, em vez de escrevê-los manualmente do zero.

O fluxo

```
┌──────────────┐    ┌────────────────┐    ┌──────────────────────┐    ┌────────────────┐
│  PROBLEMA    │───>│ META-PROMPT    │───>│PROMPT ESTRUTURADO    │───>│  EXECUÇÃO      │
│  (cenário)   │    │ (instruções    │    │(gerado pelo LLM)     │    │  (pelo LLM)    │
│              │    │  para gerar    │    │                      │    │                │
│              │    │  o prompt)     │    │                      │    │                │
└──────────────┘    └────────────────┘    └──────────────────────┘    └────────────────┘
```

Por que usar?

Consistência	Prompts gerados seguem a mesma estrutura sempre
Velocidade	Gerar um prompt estruturado leva segundos, não minutos
Expertise embut	O meta-prompt incorpora as técnicas de controle de contexto automaticame
Escalabilidade	Qualquer membro do time pode gerar prompts profissionais usando o mesmo meta-prompt
Iteração	Fácil refinar o meta-prompt e melhorar todos os prompts derivados

Meta-Prompting: Conceito (cont.)

FIAP MBA+

Quando usar

- Cenários **recorrentes** onde você precisa de prompts estruturados frequentemente
- Quando quer **padronizar** a qualidade dos prompts em um time ou organização
- Para **onboarding**: novos membros do time geram prompts profissionais desde o dia 1

Essa técnica é bastante utilizada na geração de documentos padronizados de Arquitetura, como os aderentes a frameworks como TOGAF por exemplo

Critérios de um Bom Meta-Prompt

Os 5 critérios

Um meta-prompt eficaz **instrui o LLM a gerar prompts que contenham:**

1	Definição clara do papel do modelo	Persona com expertise relevante
2	Delimitação explícita do contexto	Context Bounding + Grounding
3	Instruções para evitar alucinação	Explicit Uncertainty + Grounding
4	Formato de saída bem definido	Output Schema
5	Tratamento de incerteza	Explicit Uncertainty

Template de meta-prompt

```
Você é um especialista em prompt engineering para [DOMÍNIO].
Gere um prompt estruturado que eu possa usar para [OBJETIVO].

O prompt gerado deve:
- Definir claramente o papel do modelo (persona com expertise específica)
- Delimitar o contexto técnico e de negócio relevante
- Aplicar context bounding com escopo bem definido
- Incluir grounding com placeholders para dados concretos
- Especificar formato de saída estruturado ([JSON|Tabela|ADR|etc.])
- Exigir análise de trade-offs quando aplicável
- Instruir o modelo a indicar explicitamente incertezas e premissas
- Incluir restrições do que o modelo NÃO deve fazer
```

Critérios de um Bom Meta-Prompt (cont.)

FIAP MBA+

Dica prática

O meta-prompt funciona como um **"prompt factory"**: uma vez calibrado para seu domínio, você reutiliza sempre, mudando apenas o cenário/objetivo.

Exemplo 1: Avaliação de Migração para Cloud

FIAP MBA+

O cenário

Uma fintech de médio porte precisa avaliar se deve migrar seu datacenter on-premises para Azure nos próximos 12 meses.

Passo 1 — O que NÃO fazer (prompt genérico)

```
Me ajude a avaliar se devemos migrar para cloud.
```

Problemas:

- Sem papel definido → resposta genérica
- Sem contexto → overgeneralization
- Sem formato → resposta desestruturada
- Sem restrição → pode alucinações

Passo 2 — Meta-prompt (enviado ao LLM)

```
Você é um especialista em prompt engineering para arquitetura de soluções.
Gere um prompt estruturado que eu possa usar para avaliar uma proposta de
migração para cloud em uma empresa de médio porte. O prompt deve:
- definir claramente o papel do modelo
- delimitar o contexto técnico e de negócio relevante
- aplicar context bounding e grounding com dados concretos
- exigir análise de trade-offs (custo, latência, compliance)
- produzir saída em formato JSON com campos definidos
- instruir o modelo a indicar explicitamente quando não tiver certeza
```

Exemplo 1: Avaliação de Migração para Cloud (cont.)

Passo 3 — Prompt estruturado gerado

```
[PAPEL]: Você é um arquiteto de soluções cloud com 10 anos de
experiência em migrações enterprise. Analise apenas com base nos dados
fornecidos abaixo.

[CONTEXTO]:
- Empresa: fintech com 200 funcionários, dados regulados por LGPD
- Infraestrutura atual: datacenter on-premises, 40 VMs, Oracle DB 12c
- Workloads críticos: sistema de pagamentos (SLA 99,9%), analytics
batch noturno
- Restrições: budget máx R$ 500k/ano em cloud, equipe de 3 DevOps

[TAREFA]: Avalie a viabilidade de migração lift-and-shift para Azure
nos próximos 12 meses. Analise: custo estimado, riscos de compliance,
impacto no SLA, lacunas de competência.

[FORMATO DE SAÍDA — JSON]:
{
  "viabilidade": "alta|média|baixa",
  "custo_estimado_anual": "R$ X",
  "riscos_principais": ["risco1", "risco2"],
  "lacunas_criticas": ["gap1"],
  "recomendacao": "texto",
  "incertezas": ["ponto onde não há dados suficientes"]
}

[RESTRIÇÃO]: Se faltar informação para uma análise confiável, liste em
"incertezas" em vez de assumir dados não fornecidos.
```

Análise das técnicas aplicadas

Context Bounding	Escopo limitado a lift-and-shift, Azure, 12 meses
Grounding	Dados concretos: 40 VMs, Oracle DB, R$ 500k, 3 DevOps
Output Schema	JSON com campos definidos
Task Decomposition	4 dimensões: custo, compliance, SLA, competência
Explicit Uncertainty	Campo "incertezas" + restrição explícita

Exemplo 2: Análise de Impacto de Integração com Sistema Legado

FIAP MBA+

O cenário

Integrar um novo sistema de microserviços com um ERP SAP R/3 legado.

Prompt genérico (fraco)

```
Analise o impacto de integrar nosso sistema novo com o ERP legado.
```

Prompt estruturado (resultado de meta-prompting)

```
[PAPEL]: Você é um arquiteto de integração especializado em sistemas legados
e APIs REST. Analise apenas o cenário abaixo, sem generalizar para outros
contextos.

[CONTEXTO DO SISTEMA]:
- ERP legado: SAP R/3, interface BAPI/RFC, sem REST API nativa
- Sistema novo: microserviços em Node.js, comunicação via REST e eventos Kafka
- Volume: ~500 transações/hora no pico, janela de manutenção aos domingos 2h–4h
- Requisito: latência máx 800ms end-to-end; consistência eventual aceitável

[TAREFA]: Identifique os 3 principais riscos de integração e proponha o
padrão arquitetural mais adequado (Adapter, Anti-Corruption Layer,
Event-Driven Bridge). Justifique cada escolha com referência ao cenário acima.

[FORMATO]: Para cada risco:
  nome do risco | probabilidade (alta/média/baixa) | impacto (1-5) | mitigação
proposta

Ao final, padrão recomendado com justificativa de 3 a 5 linhas.

[RESTRIÇÃO]: Se a escolha depender de informações não fornecidas (ex: volume
futuro, budget de refatoração), sinalize explicitamente antes de prosseguir.
```

Técnicas aplicadas

Context Bounding	"Analise apenas o cenário abaixo, sem generalizar"
Grounding	SAP R/3, BAPI/RFC, 500 tx/h, 800ms latência, Kafka
Task Decomposition	3 riscos → padrão → justificativa (sequência lógica)
Output Schema	Tabela de riscos com campos definidos + parágrafo final
Explicit Uncertainty	Sinalizar lacunas antes de prosseguir

Pontos de discussão

Note que o prompt limita as opções a 3 padrões específicos (Adapter, ACL, Event-Driven Bridge). Isso é intencional: restringe o espaço de resposta para que o modelo compare opções relevantes em vez de listar todas as possibilidades do universo.

Prompts com grounding permitem que specialists validem rapidamente

Exemplo 3: Comparação de Padrões Arquiteturais para ADR

O cenário

Time precisa decidir entre Modular Monolith e Microservices para um novo produto B2B.

Prompt genérico (fraco)

```
Qual é melhor: microsserviços ou monolito?
```

Problema: esta pergunta gera um artigo genérico de blog, não uma decisão arquitetural contextualizada.

Prompt estruturado (resultado de meta-prompting)

```
[PAPEL]: Você é um arquiteto de soluções produzindo uma Architecture Decision
Record (ADR). Responda estritamente com base no cenário fornecido — não emita
opiniões sobre padrões em geral.

[CONTEXTO DA DECISÃO]:
- Produto: plataforma de gestão de contratos B2B, fase inicial (MVP em 6 meses)
- Time de desenvolvimento: 4 engenheiros full-stack, sem especialistas em DevOps
- Requisitos não-funcionais: disponibilidade 99,5%; sem exigência de escala
  independente de módulos neste momento
- Restrição: infraestrutura mínima (PaaS, sem Kubernetes)

[TAREFA]: Compare Modular Monolith vs Microservices para este cenário
específico. Produza uma ADR resumida com: contexto, opções consideradas,
decisão, consequências.
```

Técnicas aplicadas

Context Bounding	MVP, 4 engenheiros, sem K8s: escopo fechado
Grounding	Dados concretos de time, timeline e requisitos
Output Schema	Estrutura ADR padronizada com campos obrigatórios
Task Decomposition	Contexto → Opções → Decisão → Consequências
Explicit Uncertainty	"Se não se aplica ao cenário, omita" (implícito — força honestidade)

```
[FORMATO — ADR]:
Status: [Proposto]
Contexto: [2-3 linhas descrevendo o problema de decisão]
Opções consideradas:
  1. Modular Monolith — prós e contras para ESTE cenário
  2. Microservices — prós e contras para ESTE cenário
Decisão: [opção escolhida e justificativa em 3-5 linhas]
Consequências: [positivas e negativas esperadas]

[RESTRIÇÃO]: Não generalize. Se uma vantagem ou desvantagem
não se aplica ao cenário descrito, omita-a.
```

Exemplo 3: Comparação de Padrões Arquiteturais para ADR (cont.)

O que o LLM provavelmente vai recomendar

Com este contexto específico (time pequeno, MVP, sem K8s), o LLM quase certamente recomendará **Modular Monolith**, porque o grounding força o modelo a considerar as restrições reais em vez de responder com o "hype" de microserviços.

Isso é exatamente o ponto: prompts com grounding produzem recomendações **contextualizadas**, não respostas de blog.

Este exemplo é particularmente interessante porque mostra como o mesmo LLM que daria uma resposta genérica "microserviços são melhores" pode produzir uma ADR contextualizada e defensável quando recebe os dados certos

Antipadrões Comuns em Prompts

O que NÃO fazer

#	Antipadrão	Exemplo	Problema
1	**Prompt vago**	"Me ajude com minha arquitetura"	Overgeneralization — resposta genérica e inútil
2	**Excesso de contexto**	Colar 50 páginas de documentação no prompt	Context rot — modelo perde foco
3	**Sem formato definido**	"Analise e me diga o que acha"	Output imprevisível e difícil de usar
4	**Confiança cega**	Usar output do LLM direto em produção	Alucinação não detectada
5	**Delegação total**	"Decida a melhor arquitetura para mim"	LLM não deve tomar decisões — deve informar decisões
6	**Prompt único para tudo**	Mesmo prompt para migração, segurança e performance	Cada cenário precisa de um prompt específico
7	**Ignorar incerteza**	Não pedir para o modelo declarar dúvidas	Falsa confiança no output

O princípio fundamental

> *O LLM é uma **ferramenta de análise, não um tomador de decisões**. O arquiteto decide, a LLM informa, estrutura e identifica pontos cegos.*

A regra do "seria suficiente para outro humano?"

Antes de enviar um prompt, pergunte: **"Se eu desse essa mesma instrução a um consultor humano, ele teria informação suficiente para produzir uma análise útil?"**

Se a resposta for "não", o prompt precisa de mais contexto.

Quando Usar Cada Abordagem

Matriz de decisão: prompt direto vs. meta-prompting

Cenário	Abordagem recomendada	Justificativa
Pergunta pontual e simples	Prompt direto	Overhead de meta-prompting não se justifica
Análise estruturada única	Prompt direto (estruturado)	Aplicar as 5 técnicas manualmente
Cenário recorrente (ex: avaliar toda migração)	Meta-prompting	Criar uma vez, reutilizar sempre
Padronização de time	Meta-prompting	Garantir consistência entre diferentes profissionais
Cenário novo e complexo	Meta-prompting	Deixar o LLM ajudar a estruturar o prompt
Onboarding de novos membros	Meta-prompting com template	Acelerar curva de aprendizado

Quando Usar Cada Abordagem (cont)

Fluxo de decisão

```
Preciso interagir com um LLM para análise arquitetural?
│
├─ É uma pergunta simples/pontual?
│   └─ Sim → Prompt direto (mas aplique Grounding e Explicit Uncertainty)
│
├─ É uma análise estruturada?
│   ├─ Primeira vez neste tipo de cenário?
│   │   └─ Use meta-prompting para gerar o prompt
│   └─ Já tenho um prompt para cenários similares?
│       └─ Adapte o prompt existente com novo grounding
│
└─ Preciso padronizar para o time?
    └─ Crie um meta-prompt reutilizável + guia de preenchimento
```

Exercício: Análise Comparativa de Prompts

Dinâmica

Dois prompts para o **mesmo problema**: avaliação de adoção de **Event-Driven Architecture (EDA)** em uma empresa com mensageria legada e conduz a análise ao vivo com participação da turma.

Prompt A — Genérico

```
Queremos adotar event-driven architecture na nossa empresa.
Temos um sistema de mensageria antigo. O que você recomenda?
```

Prompt B — Estruturado

```
[PAPEL]: Você é um arquiteto de soluções especializado em event-driven
architecture e modernização de sistemas de mensageria. Analise exclusivamente
o cenário descrito abaixo.

[CONTEXTO]:
- Empresa: operadora de telecomunicações, 2.000 funcionários
- Sistema atual: IBM MQ Series como broker central, 15 anos em operação
- Volume: ~10.000 mensagens/hora em horário de pico
- Consumidores: 12 sistemas internos dependem do MQ para eventos de billing,
  provisioning e fault management
- Equipe: 2 especialistas em MQ (aposentando em 18 meses), 6 desenvolvedores
  Java, 3 DevOps com experiência em Kubernetes
- Restrições: zero downtime aceitável durante migração; budget de R$ 1.2M
  para 2 anos; compliance telco (Anatel)

[TAREFA]:
1. Avalie os 3 principais riscos de migrar de IBM MQ para Apache
Kafka
2. Proponha uma estratégia de migração incremental (não big-bang)
3. Identifique quais dos 12 sistemas devem ser migrados primeiro
e por quê

[FORMATO]:
Para cada risco: nome | probabilidade | impacto (1-5) | mitigação
Estratégia de migração: diagrama de fases em texto (Fase 1 → Fase
2 → Fase 3)
Priorização: tabela com sistema | criticidade | complexidade |
ordem de migração

[RESTRIÇÃO]: Se a recomendação depender de informações não
fornecidas
(ex: SLA individual de cada sistema, contratos com vendors),
sinalize
explicitamente em uma seção "Premissas e Incertezas".
```

Análise Comparativa de Prompts (cont.)

Rodada 1 — Diagnóstico do Prompt A:

> *"Olhando o Prompt A, alguém consegue identificar alguma das 5 técnicas que aprendemos no Bloco 1?"*

Resposta esperada: nenhuma.

- Context Bounding: ❌ (sem escopo)
- Grounding: ❌ ("sistema antigo" sem dados concretos)
- Output Schema: ❌ (sem formato)
- Task Decomposition: ❌ (pergunta única e aberta)
- Explicit Uncertainty: ❌ (sem instrução sobre incerteza)

Rodada 2 — Identificação no Prompt B:

> *"E no Prompt B? Me digam onde cada técnica está. Pode usar o chat."*

- Context Bounding: ✅ "Analise exclusivamente o cenário descrito abaixo"
- Grounding: ✅ IBM MQ, 10.000 msg/h, 12 sistemas, R$ 1.2M, Anatel
- Output Schema: ✅ tabela de riscos, fases, tabela de priorização
- Task Decomposition: ✅ 3 subtarefas sequenciadas
- Explicit Uncertainty: ✅ seção "Premissas e Incertezas"

Análise Comparativa de Prompts (cont.)

Rodada 3 — Melhoria coletiva (~3 min):

"Se vocês fossem o arquiteto dono desse cenário, que informação adicionariam ao Prompt B?"

Respostas esperadas:

- SLAs individuais dos 12 sistemas consumidores
- Dependências entre os sistemas (qual consome eventos de qual)
- Critérios de rollback para cada fase
- Análise de custo por fase (não só total)
- Timeline com milestones

Rodada 4 — Quando o Prompt A basta?

"Existe algum cenário em que o Prompt A é aceitável?"

Respostas esperadas:

- Brainstorming inicial (quando você ainda não tem dados)
- Pergunta conceitual ("o que é EDA?")
- Exploração informal antes de estruturar

Bons prompts sempre podem ser melhorados: é um processo iterativo. O que muda é a consciência: depois de hoje, vocês olham para um prompt e enxergam as lacunas.

Takeaways da Aula

5 pontos para levar para casa

LLMs não "sabem" mas predizem. Tratar outputs como análises, não como verdades

4 problemas fundamentais: alucinação, context rot, overgeneralization, knowledge cutoff — são estruturais, não bugs

5 técnicas de controle: Context Bounding, Grounding, Output Schema, Task Decomposition, Explicit Uncertainty: são a base de todo prompt profissional

Meta-prompting é um multiplicador de força: automatiza a criação de prompts estruturados

O arquiteto decide, o LLM informa: accountability permanece com o humano

Fórmula resumida

```
Prompt Profissional = Papel + Contexto(Grounding + Bounding)
                    + Tarefa(Decomposition) + Formato(Schema)
                    + Restrições(Uncertainty)
```

Materiais de Referência

- Anthropic: Prompt Engineering Guide (https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview)
- OpenAI: Prompt Engineering Best Practices (https://developers.openai.com/api/docs/guides/prompt-engineering)
- Google: Prompt Design Guidelines (https://cloud.google.com/discover/what-is-prompt-engineering)

OBRIGADO

/franciscojunqueira

Copyright © 2019 | Professor (a) Nome do Professor
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.

FIAP